# Project 3

# Object Oriented Programming

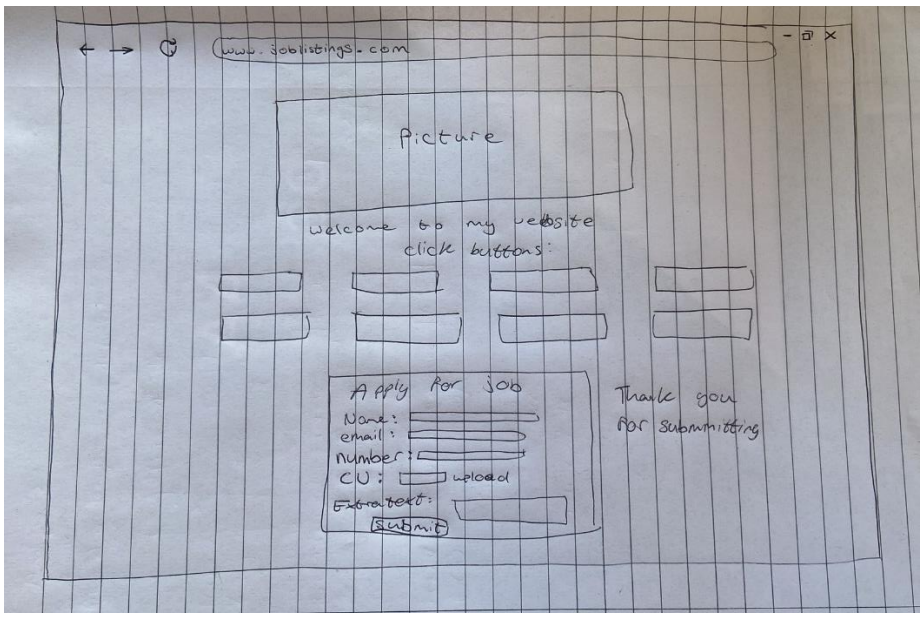
# D00255640

## Introduction

For this project, I had to use the previous data scraping project and display this data in different ways on a website. I had to use different types of files to do the different visualisations, filters, and statistics.

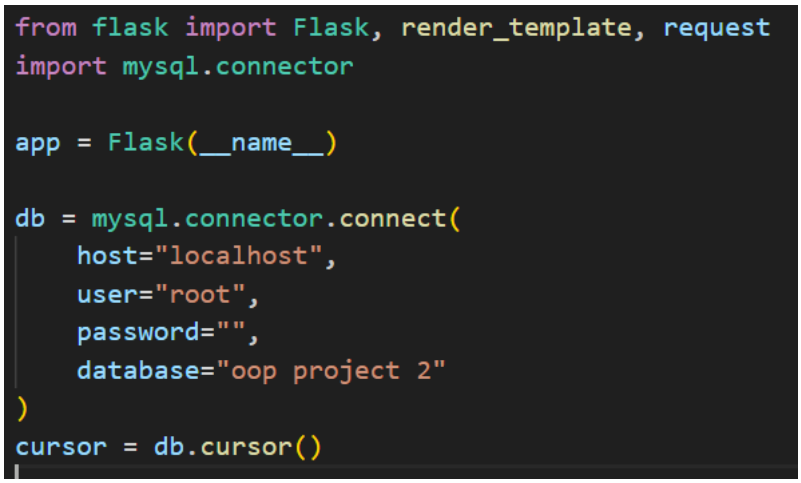
## Wire Frame

When considering the creation of my website, I wanted it to be well structured and easily navigatable. I ended up with this concept after multiple drafts. I am pleased with how my website ended up as I believe it is very similar to what I wanted to create.



# Description of HTML files

**Server.py –** My server.py file is the host for the website as the script creates a Flask web application for displaying and searching job listings. It connects to my MySQL database to retrieve job my data and provides various routes for a lot of functions such as displaying all jobs, searching for jobs based on criteria like title, company, and location, displaying job counts by location and company, and providing statistics on salaries. This file also includes functions for cleaning and analysing salary data.



```
from flask import Flask, render_template, request
import mysql.connector

app = Flask(__name__)

db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="oop project 2"
)
cursor = db.cursor()
```

**Templates folder –** my templates folder is used to contain all of my different html web pages and is used when I call the webpages on the @app.route functions from the server.py file.

- **All_jobs.html**
  This file is an HTML template designed for displaying job listings in a structured table format. The page utilizes Bootstrap for styling. Within a container div, the page presents a table with columns for job attributes such as Title, Company, Location, Salary, and Days Ago. Using a for loop, it populates the table rows with job data retrieved from the job_listings variable.

```
<body>
<div class="container">
    <h1>Job Listings</h1>
    <table class="job-table">
        <thead>
            <tr>
                <th>Title</th>
                <th>Company</th>
                <th>Location</th>
                <th>Salary</th>
                <th>Days Ago</th>
            </tr>
        </thead>
        <tbody>
            {% for row in job_listings %}
            <tr>
                <td>{{row[0]}}</td>
                <td>{{row[1]}}</td>
                <td>{{row[2]}}</td>
                <td>{{row[3]}}</td>
                <td>{{row[4]}}</td>
            </tr>
            {% endfor %}
        </tbody>
```

It interacts with the server.py file by using this get_all_job_listings function by calling the job_listings variable.

```
def get_all_job_listings():
    cursor.execute("SELECT * FROM finished")
    job_listings = cursor.fetchall()
    return job_listings
```

```
@app.route('/alljobs')
def index():
    job_listings = get_all_job_listings()
    return render_template('all_jobs.html', job_listings=job_listings)
```

- **Companycharts.html**
  The page includes the necessary JavaScript library from Chart.js CDN for chart creation. It also defines some styling for the canvas element where the chart will be displayed, ensuring it is centred and responsive. The main content of the page includes a message prompting users to hover over the bars to view the number of jobs and the corresponding company name. Inside the script tags, it checks if both companies and counts variables are available. If they are, it

creates a bar chart using Chart.js. The chart's data is populated with the companies and counts variables. The chart is rendered in a canvas element with the id "bar-chart" if the companies and counts data.

```
<body>
    <h1>Job Counts Bar Chart</h1>
    Hover on the bar to see the number of jobs and the company name
    {% if companies and counts %}
        <canvas id="bar-chart" width="800" height="450"></canvas>
        <script>
            var companies = {{ companies|tojson|safe }};
            var counts = {{ counts|tojson|safe }};

            var ctx = document.getElementById('bar-chart').getContext('2d');
            var myChart = new Chart(ctx, {
                type: 'bar',
                data: {
                    labels: companies,
                    datasets: [{
                        label: 'Counts',
                        data: counts,
                        backgroundColor: 'rgba(54, 162, 235, 0.6)',
                        borderColor: 'rgba(54, 162, 235, 1)',
                        borderWidth: 1
                    }]
```

This file interacts with the server.py file by using this code.

```
def company_with_count():
    company_counts = {}
    for row in get_all_job_listings():
        company = row[1]
        company_counts[company] = company_counts.get(company, 0) + 1
    return company_counts
```

```
@app.route('/company_counts_chart')
def display_company_counts_chart():
    company_counts = company_with_count()
    companies = [c for c in company_counts if c is not None and company_counts[c] is not None]
    counts = [company_counts[c] for c in companies]
    return render_template('companycharts.html', companies=companies, counts=counts)
```

- **Companycounts.html**
  The table contains two columns: "Company" and "Job Count." The body of the table is populated using a for loop that iterates over the company_counts dictionary, displaying each company's name and its corresponding job count in separate rows. The template presents the data in a table format, showing a summary of job counts by company.

```
<body>
    <h1>Job Counts by Company</h1>
    <table>
        <thead>
            <tr>
                <th>Company</th>
                <th>Job Count</th>
            </tr>
        </thead>
        <tbody>
            {% for company, count in company_counts.items() %}
            <tr>
                <td>{{ company }}</td>
                <td>{{ count }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</body>
```

This file interacts with my server.py file by using the previous company_with_count function and this code:

```
@app.route('/company_counts_table')
def display_company_counts_table():
    company_counts = company_with_count()
    return render_template('companycounts.html', company_counts=company_counts)
```

- **Home.html**
  This file is my home page. It contains my different html files and buttons to access them from this home page. It has a typical picture of what a job listing website would have. I also included a apply for job part, where the user inputs their information, and when “submit” button is pressed, it says “thank you for your application”. I used the javascript knowledge I learned recently in creating this part. I styled it in a way that I wanted it to look based on my wireframe.

```
<body>
    <img src="https://wrdblog.org/wp-content/uploads/2019/11/JOB-LISTINGS1-880x600.png" alt="Welcome Image"
    <h1>Welcome to my Job Listings Website</h1>
    <p>Click the buttons below to navigate:</p>
    <a href="alljobs" class="button">View All Job Listings</a>
    <a href="jobsearch" class="button">Search for Jobs</a>
    <a href="job_counts_table" class="button">View Jobs by Location (Table)</a>
    <a href="job_counts_chart" class="button">View Jobs by Location (Chart)</a>
    <a href="company_counts_table" class="button">View Jobs by Company (Table)</a>
    <a href="company_counts_chart" class="button">View Jobs by Company (Chart)</a>
    <a href="stats" class="button">View Salary Statistics</a>
</body>
```

```
<h1>Apply for a Job</h1>
<div id="thank_you_message">
    <p>Thank you for submitting your application!</p>
</div>
<p>Click the button below to apply for a job:</p>
<form id="apply_form">
    <label for="full_name">Full Name:</label>
    <input type="text" id="full_name" name="full_name" required><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required><br>

    <label for="phone">Phone Number:</label>
    <input type="text" id="phone" name="phone"><br>

    <label for="resume">Upload Resume:</label>
    <input type="file" id="resume" name="resume" accept=".pdf,.doc,.docx"><br>

    <label for="cover_letter">Cover Letter:</label>
    <textarea id="cover_letter" name="cover_letter" placeholder="Write your cover letter here"></textarea

    <input type="submit" value="Submit Application">
</form>
```

```
<script>
    document.getElementById("apply_form").addEventListener("submit", function(event) {
        event.preventDefault();
        document.getElementById("thank_you_message").style.display = "block";
    });
</script>
```

This file interacts with my server.py file by using this code:

```
@app.route('/')
def home():
    return render_template('home.html')
```

- **Jobsearch.html**
  a form with fields for "Location," "Company," and "Title," allowing users to input search criteria. Upon submitting the form, it sends a POST request to "/jobsearch" for processing. The page also displays a heading "Search Job Listings" followed by the form. Below the form, there is a section for displaying search results. If there are refined results available (refined_results), it shows them in a table format with columns for "Title," "Company," "Location," "Salary," and "Days Ago." Each row in the table corresponds to a job listing matching the search criteria. If there are no results, it displays a message "No results found."

```
<body>
    <h1>Search Job Listings</h1>
    <form action="/jobsearch" method="POST">
        <label for="location">Location:</label>
        <input type="text" id="location" name="location" value="{{ search_criteria.location }}"><br>
        <label for="company">Company:</label>
        <input type="text" id="company" name="company" value="{{ search_criteria.company }}"><br>
        <label for="title">Title:</label>
        <input type="text" id="title" name="title" value="{{ search_criteria.title }}"><br>
        <input type="submit" value="Search">
    </form>
```

```
<h2>Search Results</h2>
{% if refined_results %}
<table>
    <thead>
        <tr>
            <th>Title</th>
            <th>Company</th>
            <th>Location</th>
            <th>Salary</th>
            <th>Days Ago</th>
        </tr>
    </thead>
    <tbody>
        {% for job in refined_results %}
        <tr>
            <td>{{ job[0] }}</td>
            <td>{{ job[1] }}</td>
            <td>{{ job[2] }}</td>
            <td>{{ job[3] }}</td>
            <td>{{ job[4] }}</td>
        </tr>
        {% endfor %}
    </tbody>
</table>
{% else %}
<p>No results found.</p>
{% endif %}
```

This code interacts with the server.py file with the following code:

```
def refine_job_search_results(search_criteria):
    refined_results = []
    for row in get_all_job_listings():
        if search_criteria['title'] and search_criteria['title'].lower() not in row[0].lower():
            continue
        if search_criteria['company'] and search_criteria['company'].lower() not in row[1].lower():
            continue
        if search_criteria['location'] and search_criteria['location'].lower() not in row[2].lower():
            continue
        refined_results.append(row)
    return refined_results
```

```
@app.route('/jobsearch', methods=['GET', 'POST'])
def search_job_listings():
    search_criteria = {}
    refined_results = []

    if request.method == 'POST':
        search_criteria = {
            'location': request.form.get('location'),
            'salary': request.form.get('salary'),
            'company': request.form.get('company'),
            'title': request.form.get('title')
        }
        refined_results = refine_job_search_results(search_criteria)

    return render_template('jobsearch.html', search_criteria=search_criteria, refined_results=refined_results)
```

- **Locationcharts.html**
  After a small part of data cleaning in my clean_location function, I can remove the “Co.” part of the location and I split each value into parts as there are some locations with two locations. Through doing this I can get the accurate locations for analysis. The html code will show this location data in a bar chart.

```
<body>
    <h1>Job Counts Bar Chart</h1>
    <canvas id="bar-chart" width="800" height="450"></canvas>
    <script>
        var locations = {{ locations | tojson }};
        var counts = {{ counts | tojson }};

        var ctx = document.getElementById('bar-chart').getContext('2d');
        var myChart = new Chart(ctx, {
            type: 'bar',
            data: {
                labels: locations,
                datasets: [{
                    label: 'Job Counts by Location',
                    data: counts,
                    backgroundColor: 'rgba(54, 162, 235, 0.6)',
                    borderColor: 'rgba(54, 162, 235, 1)',
                    borderWidth: 1
                }]
            },
```

This interacts with server.py with these functions and route:

```
def clean_location(location):
    parts = [part.strip() for part in location.replace('\n', ',').split(',')]
    cleaned_parts = [part.replace('Co ', '').strip() for part in parts if part.strip()]
    return cleaned_parts[-1] if cleaned_parts else None

def location_with_count():
    location_counts = {}
    for row in get_all_job_listings():
        location = row[2]
        cleaned_location = clean_location(location)
        if cleaned_location:
            location_counts[cleaned_location] = location_counts.get(cleaned_location, 0) + 1
    return location_counts
```

```
@app.route('/job_counts_chart')
def display_job_counts_chart():
    job_counts = location_with_count()
    locations = list(job_counts.keys())
    counts = list(job_counts.values())
    return render_template('locationcharts.html', locations=locations, counts=counts)
```

- **Locationcounts.html**
  This code is similar to the previous, but the difference is that the visualisation is a table instead of a graph. This is easier to see the exact number of the jobs available in the different locations.

```
<body>
    <h1>Job Counts by Location</h1>
    <table>
        <thead>
            <tr>
                <th>Location</th>
                <th>Job Count</th>
            </tr>
        </thead>
        <tbody>
            {% for location, count in job_counts.items() %}
            <tr>
                <td>{{ location }}</td>
                <td>{{ count }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</body>
```

This file interacts with my server.py file through this code:

```
@app.route('/job_counts_table')
def display_job_counts_table():
    job_counts = location_with_count()
    return render_template('locationcounts.html', job_counts=job_counts)
```

- **Stats.html**
  For performing the statistics of the job listings data. I first had to create a new column, where the numerical values in the salary column were counted, as there was a lot of “Salary not specified” values in this column that cannot be used in the statistical analysis. This is a part of data generation.

```
<body>
    <h2>Salary Statistics</h2>
    These are the statistics for the salary data where 'Salary not specified has been removed'
    <table>
        <tr>
            <th>Statistic</th>
            <th>Value</th>
        </tr>
        <tr>
            <td>Total Count</td>
            <td>{{ total_count }}</td>
        </tr>
        <tr>
            <td>Average Salary</td>
            <td>€{{ average_salary }}</td>
        </tr>
        <tr>
            <td>Minimum Salary</td>
            <td>€{{ min_salary }}</td>
        </tr>
        <tr>
            <td>Maximum Salary</td>
            <td>€{{ max_salary }}</td>
        </tr>
    </table>
</body>
```

This file interacts with this code from the server.py file:

```python
def create_clean_salary_column():
    cursor.execute("ALTER TABLE finished ADD COLUMN clean_salary FLOAT")

def clean_salary():
    cursor.execute("UPDATE finished SET clean_salary = NULLIF(REPLACE(salary, 'Salary not specified', ''), '')")

def fetch_salary_statistics(cursor):
    cursor.execute("SELECT COUNT(clean_salary) FROM finished WHERE clean_salary IS NOT NULL")
    total_count = cursor.fetchone()[0]

    cursor.execute("SELECT AVG(clean_salary) FROM finished WHERE clean_salary IS NOT NULL")
    average_salary = cursor.fetchone()[0]

    cursor.execute("SELECT MIN(clean_salary) FROM finished WHERE clean_salary IS NOT NULL")
    min_salary = cursor.fetchone()[0]

    cursor.execute("SELECT MAX(clean_salary) FROM finished WHERE clean_salary IS NOT NULL")
    max_salary = cursor.fetchone()[0]

    return total_count, average_salary, min_salary, max_salary
```

```python
@app.route('/stats')
def display_stats():
    total_count, average_salary, min_salary, max_salary = fetch_salary_statistics(cursor)
    return render_template('stats.html', total_count=total_count, average_salary=average_salary, min_salary=min_salary, max_salary=max_sala
```

# Conclusion

After completing this project, I have a good understanding of how websites are developed and how I can use a server to store my data, and how to interact with the server to display the data in different ways on different web pages.